AEON Digital Bank

60-Minute Technical Interview Answer Key & Checklist

Java Backend Developer | Senior/Lead Level
Short, sharp, senior-sounding answers

Golden Rule: For every senior question answer with → **Concept** → **Real example** → **Trade-off** → **Failure scenario** → **Monitoring.**
For digital banking, always emphasize: **security, correctness, idempotency, transaction integrity, scalability, observability, failure handling.**

1. 60-Minute Interview Game Plan

Time	Area	Focus / Talking Points
0–5 min	Intro	12+ yrs Java, Tech Lead, Spring Boot, APIs, Kafka, K8s, AWS.
5–15 min	Project Deep-Dive	YTL/ZSmart BSS transformation; API design, challenges, prod support.
15–25 min	Java / Spring	Core Java, concurrency, Spring Boot, transactions, JPA, REST.
25–35 min	Architecture	DDD, microservices, Kafka, idempotency, Outbox, Saga, resilience.
35–45 min	DB / Security / Cloud	PostgreSQL, MongoDB, OWASP, Kubernetes, AWS.
45–55 min	Coding	1–2 practical Java problems; complexity + edge cases.
55–60 min	Assessment / Closing	Library system review, trade-offs, your questions for interviewer.

■ **Pro-tip:** Spend the first 5 min establishing credibility. Drop specific numbers: '3M+ API calls/day', 'p99 < 200ms', 'reduced latency 40%'.

2. Tell Me About Yourself — 90 Seconds

✔ **Answer:** 12+ years in Java/Java EE, mostly telecom BSS/OSS. Currently a Tech Lead — backend & API development with Spring Boot, REST, Kafka, Docker/K8s, AWS. Led the YTL BSS transformation: APIs integrating frontend with ZSmart BSS. I own design discussions, code reviews, production support, and mentor the team. I'm here because I want to bring that experience into digital banking — where correctness, security, and resilience matter most.

■ **Key Point:** Structure: Experience → Current role → Strengths → Key project → Leadership → Why this role. Don't ramble.

3. Project Deep-Dive Checklist

• **End-to-end API:** Request validation → Auth check → Business logic → Downstream call → Response/error mapping → Logging & metrics.

• **Production issue:** 'We saw p99 spike on the billing API. Root cause: connection pool exhaustion under load. Fix: increased pool, added circuit breaker, set meaningful timeouts. Monitored via Grafana dashboards.'

• **Performance improvement:** 'N+1 on loan queries → replaced with JOIN FETCH + DTO projection. Went from 120 queries per page to 1.'

• **Kafka usage:** 'Async event publishing for downstream notifications. Decouples services, handles bursts, provides at-least-once with idempotent consumers.'

• **Timeouts & retries:** 'Exponential backoff with jitter. Circuit breaker opens after 5 failures. Retry only on transient errors, never on 4xx.'

• **Separate YOUR contribution:** 'I designed the API contract and led the implementation of the core flow; the team built the UI integration.'

■ **Key Point:** Always own your part clearly. 'We did X' is vague; 'I designed and led X; the team implemented Y' is credible.

4. Java — Highest Priority

4.1 OOP + SOLID

- **SRP:** One reason to change. Example: separate LoanService from FineService.
- **OCP:** Open for extension, closed for modification. Use Strategy for new fine rules.
- **LSP:** Subtypes must be substitutable. Don't override to throw UnsupportedOperationException.
- **ISP:** Small, focused interfaces. Split a fat PaymentService into Payable, Refundable, Verifiable.
- **DIP:** Depend on abstractions. Inject FineCalculator interface, not a specific impl.

4.2 HashMap Internals

✔ **Answer:** Array of buckets (Node[]). hash(key) → bucket index. Collisions: linked list → treeified to Red-Black tree at threshold 8. Default capacity 16, load factor 0.75 → rehash to double capacity. Not thread-safe.

- **HashMap vs ConcurrentHashMap:** CHS segments the table (or uses CAS on nodes since Java 8). No full-table lock → concurrent reads/writes. Null keys/values NOT allowed in CHS.

4.3 equals() & hashCode()

✔ **Answer:** Contract: equal objects MUST have same hashcode; unequal objects SHOULD have different hashcodes. If you override equals, always override hashCode. Immutable objects (String, Integer) are safe HashMap keys. Never use mutable fields in hashCode — changing a key's field after insertion = lost entry.

4.4 Concurrency Primitives

Mechanism	When to Use	Key Point
synchronized	Simple mutual exclusion	Reentrant; intrinsic lock on object/class.
ReentrantLock	Need tryLock, fairness, or interruptible lock	Explicit lock/unlock in finally block.
volatile	Flag/shared flag visibility	Guarantees visibility, NOT atomicity.
AtomicInteger	Lock-free counters	CAS-based; no locking overhead.
StampedLock	Read-heavy with occasional writes	Optimistic read mode avoids write-blocking reads.

4.5 ExecutorService & CompletableFuture

✔ **Answer:** Never use `new Thread()`. Use ExecutorService with a bounded pool. CompletableFuture gives non-blocking composition: thenApply, thenCompose, allOf, exceptionHandle. Always supply a custom Executor — the default ForkJoinPool is shared across your app.

■ **Key Point:** Rule of thumb: CPU-bound pool size = cores; IO-bound = much higher (e.g., 50–200).

4.6 Virtual Threads

✔ **Answer:** Project Loom (Java 21+). Virtual threads are cheap — millions can run. Mounted on platform (carrier) threads. Ideal for IO-bound work (DB calls, HTTP). Don't use for CPU-bound or with synchronized blocks (pins carrier). Use with Executors.newVirtualThreadPerTaskExecutor().

4.7 Streams & Optional

✔ **Answer:** Streams: declarative data pipelines. Use map/filter/reduce/collect. Avoid side-effects in lambda. Prefer method references. Optional: never return null; use orElse/orElseThrow. Never use Optional as a field or method arg.

4.8 JVM Memory & GC

• **Heap:** Young (Eden + S0/S1) → Old. **Stack:** per-thread, frames. **Metaspace:** class metadata, native memory.
• **G1:** Default since Java 9. Region-based, predictable pause targets. Good for most workloads.
• **ZGC:** Sub-millisecond pauses, concurrent compaction. Use for latency-critical services.
• **High CPU?** Thread dump → look for tight loops or GC thrashing.
• **Memory leak?** Heap dump → MAT/VisualVM → dominator tree → biggest retained objects.
• **GC issues?** GC logs → check pause times, promotion failures, concurrent mode failures.

5. Design Patterns — MUST PREPARE

For each: **Pattern** → **When to use** → **Real example** → **Trade-off**.

Pattern	When / Why	Example in Banking / Library	Trade-off
Strategy	Interchangeable algorithms	FineCalculator with different rules per member type	More classes; but open for extension
Factory Method	Create object without exposing which impl	PaymentProcessorFactory selects card/FPX/wire	Extra indirection; but decouples client
Builder	Complex object, many optional fields	LoanDTO with optional fine, reservation ref	Verbose; but readable construction
Observer	One-to-many event notification	LoanCreated event → fine check, email, audit	Debugging event chains can be hard
Adapter	Incompatible interfaces	Legacy banking API wrapped to new internal interface	Extra layer; but protects domain from external churn
Facade	Simplify complex subsystem	LibraryService as entry point over Loan+Fine+Reservation	Can become a god class if not careful
Chain of Resp.	Pipeline of handlers	Validation chain: auth → input check → business rule	Ordering matters; debugging flow
Decorator	Add behavior dynamically	LoggingDecorator around a PaymentService	Many small classes; hard to debug stack
Template Method	Common skeleton, customizable steps	Base loan process; subclass overrides fine calc	Inheritance coupling; prefer Strategy for flexibility
Singleton	Single instance	Spring beans are singleton-scoped by default	Global state; testability; use Spring's container

✔ **Answer: Practical question: 'Three payment types with different logic.'** Use Strategy + Factory. Factory picks the correct Strategy bean based on payment type. Each strategy owns its own processing. Open/Closed: add a new payment type → add a new Strategy impl, no existing code changes.

■ **Key Point:** Also know: Repository, DI, MVC, CQRS, Saga, Outbox, Circuit Breaker, API Gateway — these are architectural patterns, not just GoF.

6. Spring Boot / JPA / Transactions

6.1 IoC / DI / Bean Lifecycle

✔ **Answer:** Spring container creates, wires, and manages beans. Lifecycle: instantiation → @PostConstruct → business use → @PreDestroy. Scopes: singleton (default), prototype, request, session. Auto-configuration: @ConditionalOnClass/MissingBean → sensible defaults.

6.2 AOP & Proxy-Based @Transactional

✔ **Answer:** Spring creates a CGLIB (or JDK dynamic) proxy around @Transactional beans. The proxy opens a transaction BEFORE the method call and commits/rolls back AFTER. This means **self-invocation bypasses the proxy** — a method calling another @Transactional method via `this` will NOT start a new transaction. Private/final methods also can't be proxied.

■ **Key Point:** Self-invocation is the #1 reason @Transactional 'doesn't work' in interviews.

6.3 Propagation & Isolation

Propagation	Meaning
REQUIRED (default)	Join existing tx; start new if none.
REQUIRES_NEW	Always start a new tx; suspend outer.
NESTED	Savepoint within outer tx; rollback to savepoint on error.
SUPPORTS / NOT_SUPPORTED / NEVER / MANDATORY	Edge cases — know the names.

• **Isolation levels:** READ_UNCOMMITTED → READ_COMMITTED (default in PG) → REPEATABLE_READ → SERIALIZABLE. Higher = safer but slower. Dirty read → Non-repeatable read → Phantom read.

• **Rollback:** Checked exceptions do NOT trigger rollback by default. Use rollbackFor = Exception.class to be safe.

6.4 JPA: Lazy vs EAGER, N+1, Locking

✔ **Answer: Lazy (default for @OneToMany):** loads on access. Must be within session/tx. **EAGER:** loads immediately. N+1: JPA issues 1 query for parent + N for children. Fix: JOIN FETCH in query, or @EntityGraph, or DTO projection with Spring Data Projections.

• **Optimistic locking:** @Version field. On update, JPA checks version matches. If not → ObjectOptimisticLockingFailureException. No DB lock. Good for low contention.

• **Pessimistic locking:** SELECT … FOR UPDATE. DB row lock. Good for high contention (e.g., borrowing the last available copy of a book).

6.5 REST Validation & Exception Handling

✔ **Answer:** @Valid + Bean Validation (@NotNull, @Size, @Pattern) on request DTOs. Global @ControllerAdvice + @ExceptionHandler catches all exceptions, returns consistent error response. Never expose stack traces to clients.

7. Microservices + DDD

7.1 DDD Key Terms

Term	Definition
Domain	The business problem space.
Subdomain	A coherent area within the domain (Core, Supporting, Generic).
Bounded Context	Linguistic & model boundary; same term, different meaning across contexts.
Entity	Has identity that persists over time (e.g., Loan).
Value Object	No identity; defined by attributes (e.g., Money, Address).
Aggregate	Cluster of entities + value objects treated as one unit.
Aggregate Root	The entry entity; external access only through it.
Domain Event	Something that happened in the domain (e.g., LoanCreated).
Repository	Persistence abstraction; one per aggregate root.

7.2 Microservice Principles

- **Service boundaries:** Align with business capabilities, not technical layers.
- **DB-per-service:** Each service owns its data. No direct DB access across services.
- **Sync vs Async:** REST/gRPC for request-response. Kafka for domain events & decoupling.
- **Resilience:** Circuit breaker (stop calling failing service), timeout (fail fast), retry with backoff (transient errors), bulkhead (isolate resource pools).
- **Eventual consistency:** Across services, we accept temporary inconsistency. Sagas coordinate; outbox ensures reliable publishing.

7.3 Saga: Orchestration vs Choreography

✔ **Answer: Orchestration:** Central coordinator tells each service what to do. Easier to track, single point of failure. **Choreography:** Each service reacts to events. More decoupled, harder to trace. For banking: prefer orchestration — audit trail matters.

7.4 Outbox Pattern

✔ **Answer:** Problem: DB commit + Kafka publish must be atomic, but they aren't. Solution: Write to an Outbox table in the SAME DB transaction. A separate relay (Debezium CDC or polling) reads the outbox and publishes to Kafka. Guaranteed: if the tx commits, the event WILL be published.

8. Kafka / Event-Driven Architecture

Concept	Key Point
Topic / Partition	Topic = logical stream; partition = ordered, parallel unit.
Offset	Consumer's position; committed periodically for at-least-once.
Consumer Group	Each partition consumed by ONE consumer in a group. Scale = partition count.
Ordering	Guaranteed WITHIN a partition only. Use partition key (e.g., accountId).
At-least-once	Default. Consumer commits after processing → crash before commit = reprocess.
Idempotent consumer	Use idempotency key + DB unique constraint. Duplicate = no side effect.
Retry + DLQ	Failed message → retry topic with backoff → after N retries → Dead Letter Topic.
Producer idempotence	enable.idempotence=true; Kafka deduplicates on (PID, seq#).
Transactions	Consumer→process→produce + commit offset in one Kafka tx.

✔ **Answer: Interview answer:** 'We use Kafka for async domain events. Partition by aggregate ID for ordering. Consumers are idempotent via unique constraint on event ID. Failed messages go to a retry topic with exponential backoff, then DLQ for manual review. Outbox pattern ensures DB + Kafka atomicity.'

9. PostgreSQL + MongoDB

9.1 PostgreSQL

• **ACID:** Atomicity, Consistency, Isolation, Durability — the gold standard for financial data.

• **Indexes:** B-tree (default), GIN (full-text), partial, composite. Composite index column order matters — most selective first, or match query patterns.

• **EXPLAIN ANALYZE:** Always use this to verify query plans. Look for Seq Scan, index usage, join strategy.

• **Connection pool:** HikariCP. Pool exhaustion = app hangs. Monitor active/idle/pending connections.

• **Locking:** Row-level (FOR UPDATE), table-level, advisory locks. Deadlocks detected; transaction rolls back.

• **Optimistic:** @Version column. Pessimistic: SELECT FOR UPDATE. Choose by contention level.

9.2 MongoDB

• **Document model:** Embed for 1-to-few; reference for 1-to-many/many-to-many.

• **Indexes:** Single, compound, text, geospatial. Covered queries = index only, no document fetch.

• **Aggregation:** Pipeline stages: $match → $group → $sort → $project.

• **Replica set:** Primary + secondaries. Auto-failover. Reads from secondary = eventual consistency.

• **Sharding:** Horizontal scale. Shard key determines data distribution. Choose wisely — can't change easily.

✔ **Answer: When to choose what:** PostgreSQL for strongly relational, transactional banking data (accounts, transfers, loans). MongoDB for document-oriented, flexible-schema domains (audit logs, config, analytics events). Often both coexist.

10. Security — OWASP + Digital Banking

• **OWASP Top 10:** Broken Access Control, Crypto Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable Components, Auth Failures, Integrity Failures, Logging/Monitoring Failures, SSRF.

Must-Know Security Points

• **HTTPS/TLS everywhere.** Mutual TLS for service-to-service in zero-trust architectures.

• **OAuth2/OIDC:** Authorization Server issues JWT. Resource Server validates. Short-lived access tokens + refresh tokens. Rotate signing keys.

• **AuthN ≠ AuthZ:** Authentication = who are you? Authorization = what can you do? Always enforce business-level authorization, not just role checks.

• **Input validation:** Validate on boundary. Parameterized queries / JPA = no SQL injection.

• **Never log:** passwords, tokens, secrets, unnecessary PII. Mask account numbers.

• **Secrets management:** Kubernetes Secrets (base64, not encryption by default) or AWS Secrets Manager + IAM least-privilege policies.

• **Rate limiting:** Protect APIs from abuse. Audit logs for every financial operation.

• **Idempotency + transaction integrity:** Every money-movement request carries an idempotency key. DB unique constraint prevents double-processing. If in doubt, reject — never double-charge.

11. Kubernetes / Docker / AWS / CI-CD

11.1 Kubernetes

Concept	Key Point
Pod	Smallest deployable unit; 1+ containers sharing network/storage.
Deployment	Declarative updates, replicas, rolling strategy.
Service	Stable network endpoint; routes to pods via labels.
ConfigMap / Secret	External config; Secret = base64-encoded (use external KMS for real encryption).
Liveness probe	Is the app healthy? Fails → restart pod.
Readiness probe	Can the app serve traffic? Fails → remove from Service.
CrashLoopBackOff	Check: kubectl describe pod, logs, previous-logs, events, probes, config.
OOMKilled	Memory limit too low; JVM heap + native memory > limit. Set -Xmx < limit.

11.2 Docker Best Practices

- Multi-stage build: build in one image, copy JAR to slim runtime image.
- Use distroless or eclipse-temurin-alpine. Run as non-root user.
- Vulnerability scanning in CI (Trivy, Snyk). Pin base image digests.

11.3 CI/CD Pipeline

Build → Unit tests → SonarQube scan → Build Docker image → Push to ECR → Deploy to EKS (canary/blue-green) → Smoke tests → Promote or rollback.

11.4 AWS Essentials

- **EKS:** Managed Kubernetes. **RDS:** Managed PostgreSQL. **ECR:** Container registry. **ALB:** Load balancer. **IAM:** Fine-grained roles. **CloudWatch:** Logs + metrics + alarms. **Secrets Manager:** Encrypted secrets with auto-rotation. **VPC:** Network isolation — public subnets (ALB), private (EKS, RDS).

12. Library Management System Assessment

Requirements: books, copies, members, search, borrow, return, due dates, fines, reservations, availability.

Entity Model

Entity	Key Fields	Notes
Book	isbn, title, author	The catalog entry (title). Not the physical copy.
BookCopy	id, book_id, status	Individually trackable physical copy. Status: AVAILABLE/BORROWED/LOST.
Member	id, name, email, status	ACTIVE/SUSPENDED.
Loan	id, copy_id, member_id, borrowDate, dueDate, returnDate	Borrowing record.
Reservation	id, book_id, member_id, status	WAITING/FULFILLED/CANCELLED.
Fine	id, loan_id, amount, status	UNPAID/PAID.

API Endpoints

- POST /books — Add a book (and optionally copies).
- GET /books — Search books (title, author, ISBN).
- POST /members — Register a member.
- POST /loans — Borrow a copy. **Lock the BookCopy row** (SELECT FOR UPDATE) to prevent double-borrow. Check status=AVAILABLE first.
- POST /loans/{id}/return — Return a copy. Calculate fine if overdue. Update copy status.
- POST /reservations — Reserve a book when no copy is available.

Concurrency: The Critical Interview Point

✔ **Answer:** Two members borrowing the same available copy: use @Transactional + pessimistic lock (SELECT FOR UPDATE on BookCopy row). This serializes access. Alternatively, optimistic locking with @Version — but under high contention, you'll get many retry failures. For a library, pessimistic is simpler and correct.

■ **Key Point:** Lock the BookCopy row, NOT the Loan row. The invariant is on the copy's availability.

Pattern Choices

- **Strategy → FineCalculator:** Different fine rules per member type or loan period. Inject FineCalculator; each impl computes differently. Open for extension.
- **Factory → NotificationSender:** Email vs SMS vs push. Factory picks the right sender.
- **Repository → Persistence:** Spring Data JPA repositories. Abstraction over data access.
- **Facade → LibraryService:** Single entry point coordinating Loan, Fine, Reservation. Keeps controllers thin.

■ **Pro-tip:** Don't over-engineer. Explain WHY each pattern is needed. 'We use Strategy here because fine rules vary and will change — open/closed principle.'

13. Coding Practice

- **HashMap frequency:** Iterate, count with map.getOrDefault(k,0)+1. O(n).
- **First non-repeating char:** Two-pass. First: count frequency. Second: find first with count 1. O(n).
- **Two Sum:** HashMap of value→index. For each element, check if complement exists. O(n).
- **Sliding window:** Max subarray of size k. Maintain window sum, slide, update max. O(n).
- **Merge intervals:** Sort by start. Iterate, merge if overlapping. O(n log n).
- **LRU Cache:** HashMap + doubly linked list. Get/Put in O(1). Evict tail on overflow.
- **Thread-safe counter:** AtomicInteger.incrementAndGet(). Or LongAdder for high contention.

✔ **Answer: Coding flow:** Clarify requirements → Approach (name data structures) → Code → Test edge cases (null, empty, single element, duplicates) → State time/space complexity.

■ **Key Point:** For Library assessment coding: Borrow/Return with clean OOP. Show you think about concurrency and error handling, not just happy path.

14. Production Scenarios

p99 doubled, CPU flat

✔ **Answer:** Not CPU-bound. Check: DB response times, connection pool saturation, thread pool queue, lock contention, network latency, external API slowdown, Kafka consumer lag, GC pauses. Use APM traces (Jaeger/Datadog) to find the slow span.

API returning 500s

✔ **Answer:** Check logs for exceptions. Was there a recent deployment? Check downstream health (DB, Kafka, external APIs). Check metrics: error rate, latency, connection pool. Roll back if deployment-caused. Fix forward if known.

Kafka consumer lag growing

✔ **Answer:** Is consumer healthy? Check processing time per message → is a downstream call slow? Are partitions balanced? Scale consumers (can't exceed partition count). Check for poison pill messages blocking the thread.

DB CPU high

✔ **Answer:** Find slow queries: pg_stat_statements, EXPLAIN ANALYZE. Missing indexes? Full table scans? Check connection count, lock waits, long-running transactions.

Transaction not rolling back

✔ **Answer:** 1) Throwing a checked exception (default rollback is unchecked only). 2) Self-invocation bypassing proxy. 3) Wrong propagation (SUPPORTS has no tx). 4) @Transactional on a private method (proxy can't intercept). Fix: rollbackFor = Exception.class.

Pod restarts / OOMKilled

✔ **Answer:** Check kubectl describe pod, events, previous logs. OOMKilled: JVM heap (-Xmx) too close to container memory limit. Rule: set -Xmx to ~70–80% of limit. Also check native memory (off-heap, NIO, metaspace).

Duplicate payment/event

✔ **Answer:** Idempotency key on every request. DB unique constraint on (idempotency_key). If duplicate: return cached result, don't reprocess. Consumer side: deduplicate by event ID + processed flag in DB.

15. Leadership / Ownership

• **Mentoring:** Pair programming, weekly 1:1, code review as teaching moment. I create ADRs (Architecture Decision Records) so the team understands the ‘why’.

• **Code/Design reviews:** Check correctness first, then readability, then patterns. I ask ‘what happens if this fails?’ rather than just style nitpicks.

• **Architecture disagreements:** Write an ADR with options, trade-offs, and a recommendation. Data beats opinions. If still unresolved, time-box the decision and commit.

• **Tech debt vs delivery:** Track debt as visible backlog items. Allocate 20% sprint capacity. Never let debt silently accumulate — make it visible, make it a choice.

• **Communicating to non-tech:** Frame in business impact. ‘This change reduces incident response time by 50%’ beats ‘we’re adding circuit breakers.’

• **Production incident I owned:** Describe using STAR: Situation → Task → Action → Result. Emphasize: detected fast, mitigated, root-caused, prevented recurrence.

■ **Key Point:** Be honest: if you’re a technical lead (not people manager), say so. Mentoring ≠ managing. Both are leadership.

16. Final 10-Minute Checklist

Tick each item. If you can’t answer in 30 seconds, review that section.

- ■ HashMap internals & HashMap vs ConcurrentHashMap
- ■ @Transactional: propagation, isolation, self-invocation pitfall
- ■ N+1 problem: cause, fix (JOIN FETCH / DTO projection)
- ■ Optimistic vs pessimistic locking
- ■ Design Patterns: Strategy, Factory, Builder, Adapter, Observer, Facade
- ■ DDD: bounded context, aggregate root, domain event
- ■ Kafka: consumer groups, partition ordering, idempotent consumer, DLQ
- ■ Outbox pattern + Saga (orchestration vs choreography)
- ■ PostgreSQL vs MongoDB: when to choose which
- ■ OWASP Top 3 + OAuth2/JWT flow + AuthN vs AuthZ
- ■ Kubernetes: liveness vs readiness, OOMKilled, CrashLoopBackOff
- ■ Library Management System: entities, borrow concurrency, pattern choices
- ■ One production incident story + one performance improvement story
- ■ 90-second introduction rehearsed and timed